



আন্তর্জাতিক ইসলামি বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتاغونغ
International Islamic University Chittagong

LAB REPORT

EXPERIMENT NO : 01
EXPERIMENT NAME : Implementation of SQL Queries for Data Retrieval
COURSE CODE : CSE-2424
COURSE TITLE : Database Management Systems Lab

SUBMITTED TO :

Mr. Mohammad Sakif Ibn Yousuf

Lecturer,
Dept. of CSE, IIUC

SUBMITTED BY :

NAME : Md. Shahraj Mashrafe Mashfe
ID NO : C243217
SEMESTER : 4th
SECTION : 4FM
DEPARTMENT : CSE

REMARK:

DATE OF SUBMISSION : 2026-03-29



Database Initialization and Data Entry :

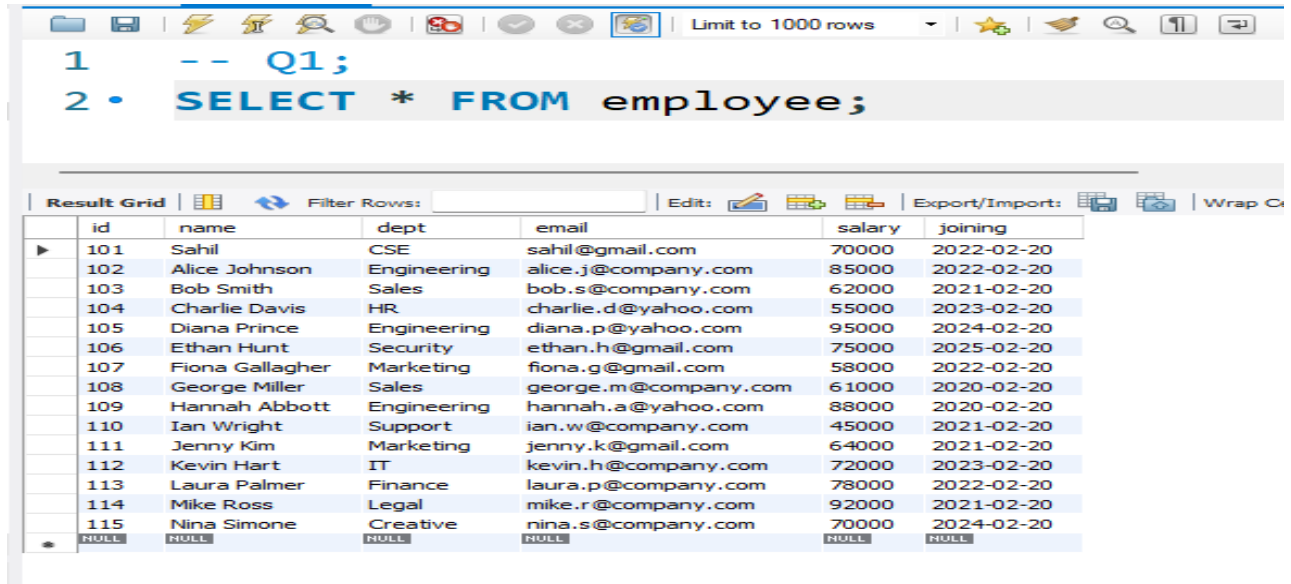
- i. First, I initialized the table using CREATE TABLE, where I defined all columns like id, name, dept, email, salary, and joining date with appropriate data types.
- ii. I set the ID as the primary key to identify each employee uniquely.
- iii. Then, I inserted data into the table using INSERT INTO statements.
- iv. Each row was added with proper values matching the column order and correct formats (especially dates in YYYY-MM-DD).
- v. This setup ensured the table was structured correctly and ready for running different SQL queries.

```
1  -- CREATE DATABASE C243217_A01;
2
3  -- USE C243217_A01;
4
5  -- CREATE TABLE employee(
6  -- id INT AUTO_INCREMENT PRIMARY KEY,
7  -- name VARCHAR(25) NOT NULL,
8  -- dept VARCHAR(20) NOT NULL,
9  -- email VARCHAR(2) UNIQUE,
10 -- salary INT
11 -- );
12
13 -- ALTER TABLE employee ADD COLUMN joining DATE;
14
15 -- ALTER TABLE employee MODIFY COLUMN email VARCHAR(30) UNIQUE;
16
17 • INSERT INTO employee(id, name, dept, email, salary, joining)
18
19 VALUES
20 (101, 'Sahil', 'CSE', 'sahil@gmail.com', 70000, '2022-02-20'),
21 (102, 'Alice Johnson', 'Engineering', 'alice.j@company.com', 85000, '2022-02-20'),
22 (103, 'Bob Smith', 'Sales', 'bob.s@company.com', 62000, '2021-02-20'),
23 (104, 'Charlie Davis', 'HR', 'charlie.d@yahoo.com', 55000, '2023-02-20'),
24 (105, 'Diana Prince', 'Engineering', 'diana.p@yahoo.com', 95000, '2024-02-20'),
25 (106, 'Ethan Hunt', 'Security', 'ethan.h@gmail.com', 75000, '2025-02-20'),
26 (107, 'Fiona Gallagher', 'Marketing', 'fiona.g@gmail.com', 58000, '2022-02-20'),
27 (108, 'George Miller', 'Sales', 'george.m@company.com', 61000, '2020-02-20'),
28 (109, 'Hannah Abbott', 'Engineering', 'hannah.a@yahoo.com', 88000, '2020-02-20'),
29 (110, 'Ian Wright', 'Support', 'ian.w@company.com', 45000, '2021-02-20'),
30 (111, 'Jenny Kim', 'Marketing', 'jenny.k@gmail.com', 64000, '2021-02-20'),
31 (112, 'Kevin Hart', 'IT', 'kevin.h@company.com', 72000, '2023-02-20'),
32 (113, 'Laura Palmer', 'Finance', 'laura.p@company.com', 78000, '2022-02-20'),
33 (114, 'Mike Ross', 'Legal', 'mike.r@company.com', 92000, '2021-02-20'),
34 (115, 'Nina Simone', 'Creative', 'nina.s@company.com', 70000, '2024-02-20');
```

Q1: Display all records from the Employee table

I selected all columns using SELECT *.

No conditions were applied since I needed the full table.



The screenshot shows a database query editor with a toolbar at the top. The SQL query is entered in the editor area:

```
1 -- Q1;  
2 • SELECT * FROM employee;
```

Below the query editor is the 'Result Grid' tab, which displays the results of the query. The grid has columns for 'id', 'name', 'dept', 'email', 'salary', and 'joining'. The data is as follows:

	id	name	dept	email	salary	joining
▶	101	Sahil	CSE	sahil@gmail.com	70000	2022-02-20
	102	Alice Johnson	Engineering	alice.j@company.com	85000	2022-02-20
	103	Bob Smith	Sales	bob.s@company.com	62000	2021-02-20
	104	Charlie Davis	HR	charlie.d@yahoo.com	55000	2023-02-20
	105	Diana Prince	Engineering	diana.p@yahoo.com	95000	2024-02-20
	106	Ethan Hunt	Security	ethan.h@gmail.com	75000	2025-02-20
	107	Fiona Gallagher	Marketing	fiona.g@gmail.com	58000	2022-02-20
	108	George Miller	Sales	george.m@company.com	61000	2020-02-20
	109	Hannah Abbott	Engineering	hannah.a@yahoo.com	88000	2020-02-20
	110	Ian Wright	Support	ian.w@company.com	45000	2021-02-20
	111	Jenny Kim	Marketing	jenny.k@gmail.com	64000	2021-02-20
	112	Kevin Hart	IT	kevin.h@company.com	72000	2023-02-20
	113	Laura Palmer	Finance	laura.p@company.com	78000	2022-02-20
	114	Mike Ross	Legal	mike.r@company.com	92000	2021-02-20
	115	Nina Simone	Creative	nina.s@company.com	70000	2024-02-20
	NULL	NULL	NULL	NULL	NULL	NULL

Q2: Display only employee names and salaries

I specified only the name and salary in the SELECT clause.

This limits the output to just those two fields.



The screenshot shows a database query editor with a toolbar at the top. The SQL query is entered in the editor area:

```
1 -- Q2;  
2  
3 • SELECT name, salary FROM employee;
```

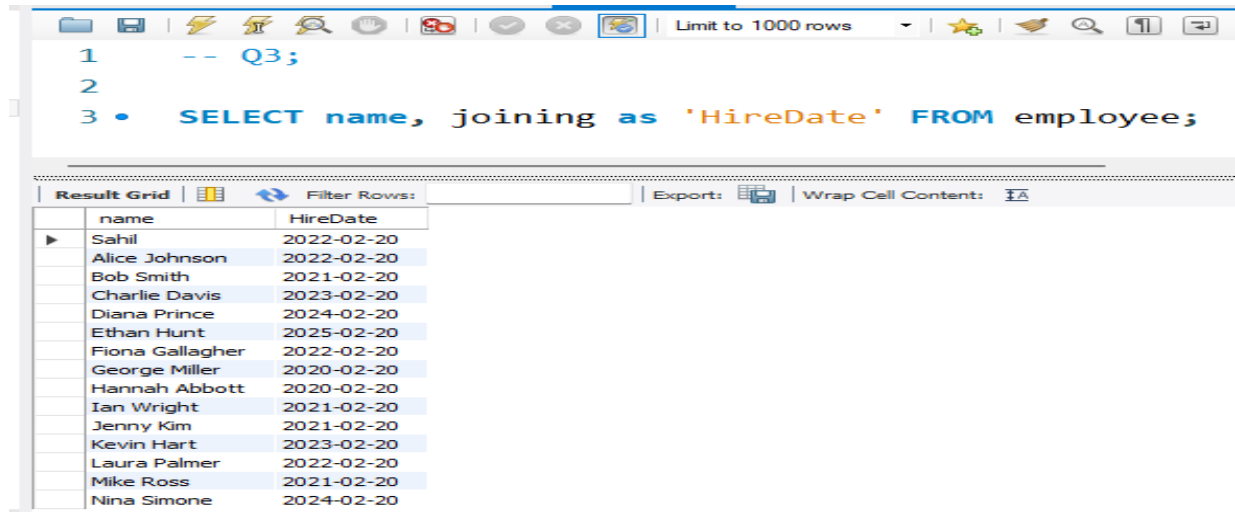
Below the query editor is the 'Result Grid' tab, which displays the results of the query. The grid has columns for 'name' and 'salary'. The data is as follows:

	name	salary
▶	Sahil	70000
	Alice Johnson	85000
	Bob Smith	62000
	Charlie Davis	55000
	Diana Prince	95000
	Ethan Hunt	75000
	Fiona Gallagher	58000
	George Miller	61000
	Hannah Abbott	88000
	Ian Wright	45000
	Jenny Kim	64000
	Kevin Hart	72000
	Laura Palmer	78000
	Mike Ross	92000
	Nina Simone	70000

Q3: Display employee name and joining date with a custom column name

I selected the name and the joining date.

Then I used AS to rename the joining date to a custom heading.



The screenshot shows a SQL query editor with the following query:

```
-- Q3;  
  
SELECT name, joining as 'HireDate' FROM employee;
```

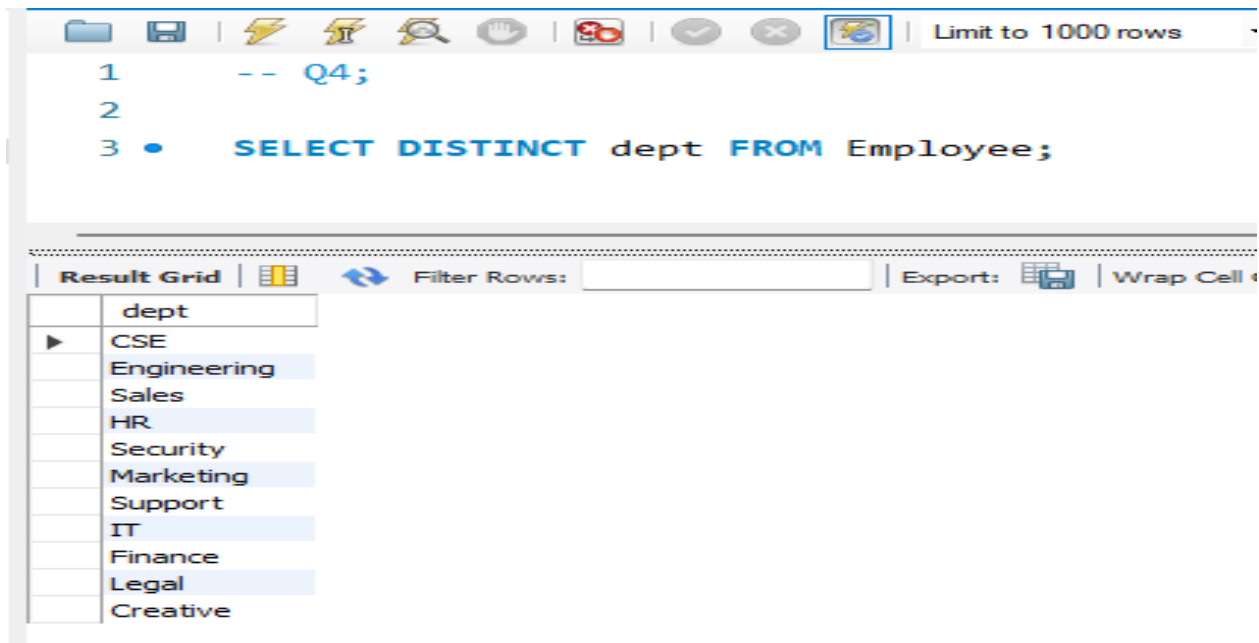
Below the query editor is a "Result Grid" showing the results of the query. The grid has two columns: "name" and "HireDate". The results are as follows:

name	HireDate
Sahil	2022-02-20
Alice Johnson	2022-02-20
Bob Smith	2021-02-20
Charlie Davis	2023-02-20
Diana Prince	2024-02-20
Ethan Hunt	2025-02-20
Fiona Gallagher	2022-02-20
George Miller	2020-02-20
Hannah Abbott	2020-02-20
Ian Wright	2021-02-20
Jenny Kim	2021-02-20
Kevin Hart	2023-02-20
Laura Palmer	2022-02-20
Mike Ross	2021-02-20
Nina Simone	2024-02-20

Q4: Display unique department names

I used SELECT DISTINCT dept to remove duplicates.

This ensures each department appears only once.



The screenshot shows a SQL query editor with the following query:

```
-- Q4;  
  
SELECT DISTINCT dept FROM Employee;
```

Below the query editor is a "Result Grid" showing the results of the query. The grid has one column: "dept". The results are as follows:

dept
CSE
Engineering
Sales
HR
Security
Marketing
Support
IT
Finance
Legal
Creative

Q5: Display employees from the IT department

I filtered rows using WHERE dept = 'IT'.

This returns only employees from the IT department.

```
1  -- Q5;
2
3 •  SELECT * FROM employee WHERE dept = 'IT';
```

Result Grid   Filter Rows: Edit:    Export/Import: 						
	id	name	dept	email	salary	joining
▶	112	Kevin Hart	IT	kevin.h@company.com	72000	2023-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q6: Display employees with salary greater than 75000

I applied a condition salary > 75000.

This shows employees earning more than the given amount.

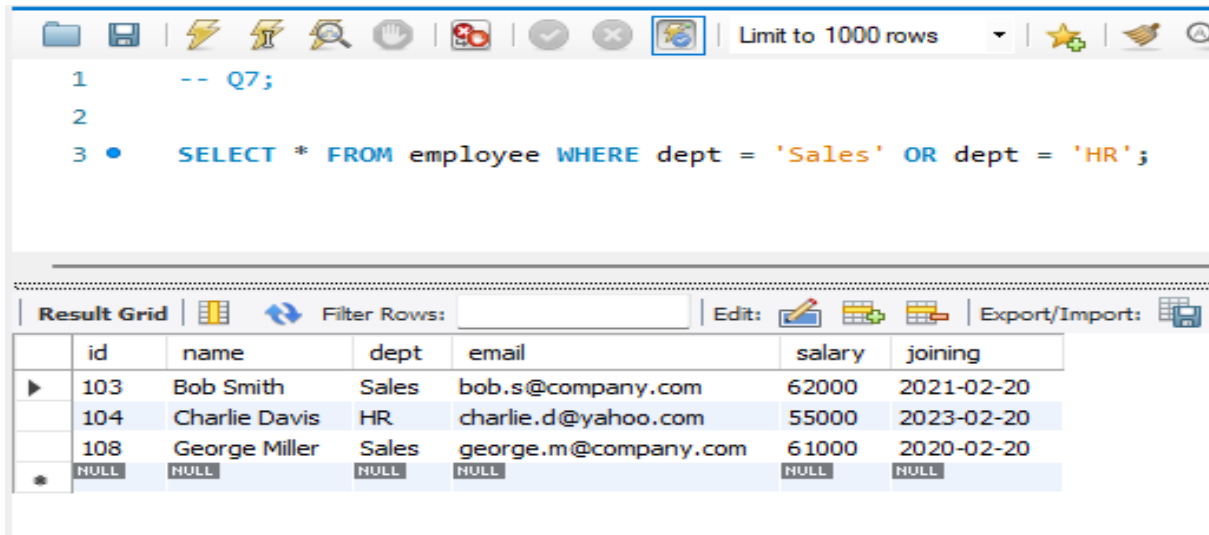
```
1  -- Q6;
2
3 •  SELECT * FROM employee WHERE salary > 75000;
```

Limit to 1000 rows	Filter Rows:	Edit:	Export/Import:	Wrap Cel		
	id	name	dept	email	salary	joining
▶	102	Alice Johnson	Engineering	alice.j@company.com	85000	2022-02-20
	105	Diana Prince	Engineering	diana.p@yahoo.com	95000	2024-02-20
	109	Hannah Abbott	Engineering	hannah.a@yahoo.com	88000	2020-02-20
	113	Laura Palmer	Finance	laura.p@company.com	78000	2022-02-20
	114	Mike Ross	Legal	mike.r@company.com	92000	2021-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q7: Display employees from Sales and HR departments

I used WHERE dept IN ('Sales', 'HR').

This filters employees working in either of the two departments.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The query editor contains the following SQL code:

```
1  -- Q7;  
2  
3  •  SELECT * FROM employee WHERE dept = 'Sales' OR dept = 'HR';
```

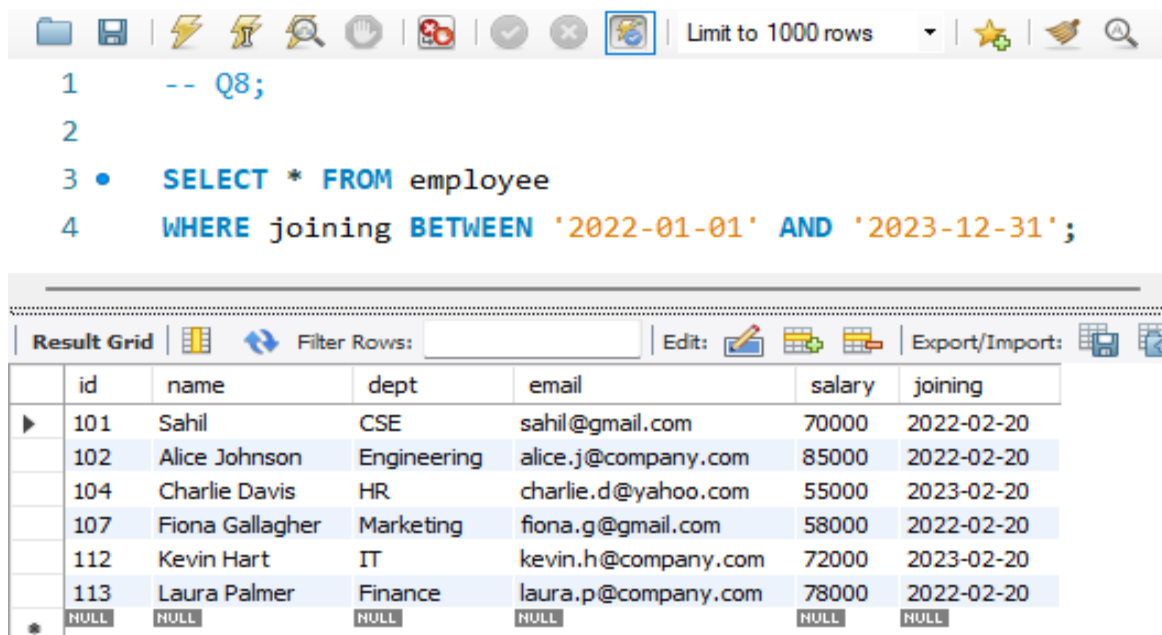
Below the query editor is the 'Result Grid' tab. It displays a table with 7 columns: id, name, dept, email, salary, and joining. The table contains 4 rows of data, with the last row being a summary row with NULL values.

	id	name	dept	email	salary	joining
▶	103	Bob Smith	Sales	bob.s@company.com	62000	2021-02-20
	104	Charlie Davis	HR	charlie.d@yahoo.com	55000	2023-02-20
	108	George Miller	Sales	george.m@company.com	61000	2020-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q8: Display employees hired between 2022 and 2023

I used BETWEEN '2022-01-01' AND '2023-12-31'.

This selects employees hired within that date range.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The query editor contains the following SQL code:

```
1  -- Q8;  
2  
3  •  SELECT * FROM employee  
4  WHERE joining BETWEEN '2022-01-01' AND '2023-12-31';
```

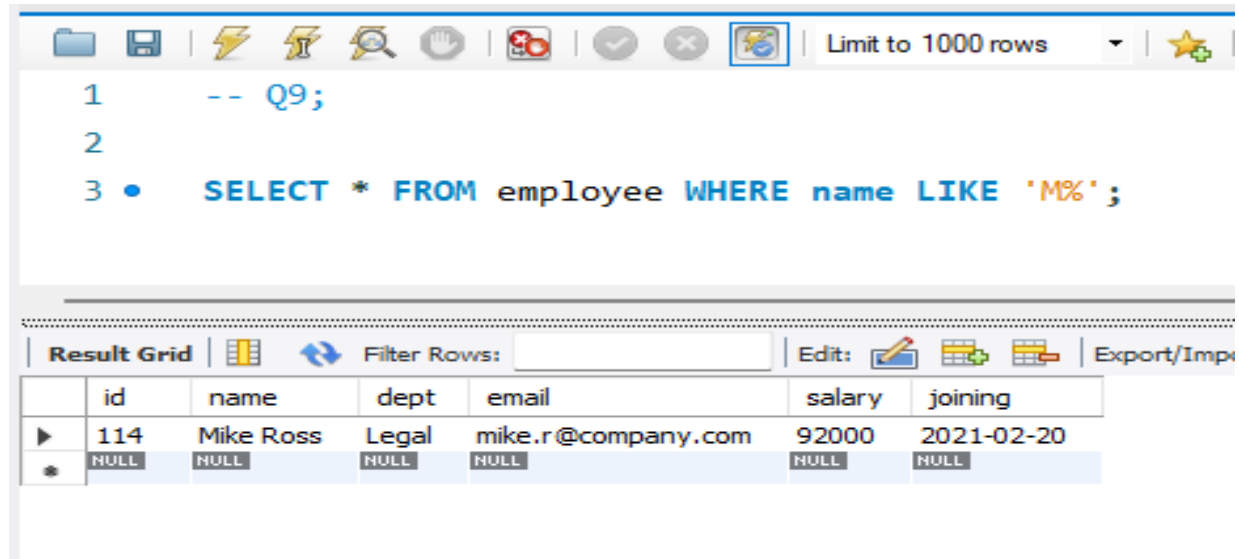
Below the query editor is the 'Result Grid' tab. It displays a table with 7 columns: id, name, dept, email, salary, and joining. The table contains 7 rows of data, with the last row being a summary row with NULL values.

	id	name	dept	email	salary	joining
▶	101	Sahil	CSE	sahil@gmail.com	70000	2022-02-20
	102	Alice Johnson	Engineering	alice.j@company.com	85000	2022-02-20
	104	Charlie Davis	HR	charlie.d@yahoo.com	55000	2023-02-20
	107	Fiona Gallagher	Marketing	fiona.g@gmail.com	58000	2022-02-20
	112	Kevin Hart	IT	kevin.h@company.com	72000	2023-02-20
	113	Laura Palmer	Finance	laura.p@company.com	78000	2022-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q9: Display employees whose names start with 'M'

I used LIKE 'M%' to match names starting with M.

The % acts as a wildcard for remaining characters.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1  -- Q9;  
2  
3  •  SELECT * FROM employee WHERE name LIKE 'M%';
```

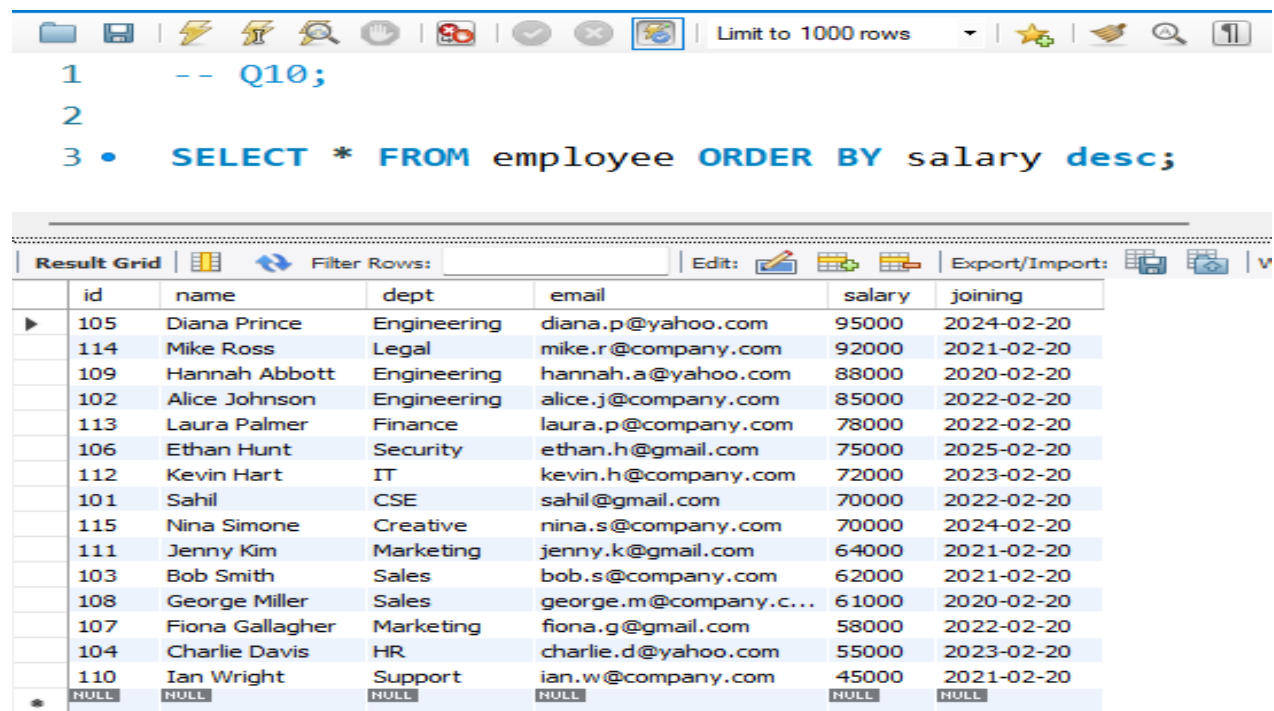
Below the query editor, the 'Result Grid' is displayed. It shows a table with 7 columns: id, name, dept, email, salary, and joining. The first row is highlighted, showing the results for Mike Ross.

	id	name	dept	email	salary	joining
▶	114	Mike Ross	Legal	mike.r@company.com	92000	2021-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q10: Display employees sorted by salary (highest first)

I sorted results using ORDER BY salary DESC.

This arranges employees from highest to lowest salary.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1  -- Q10;  
2  
3  •  SELECT * FROM employee ORDER BY salary desc;
```

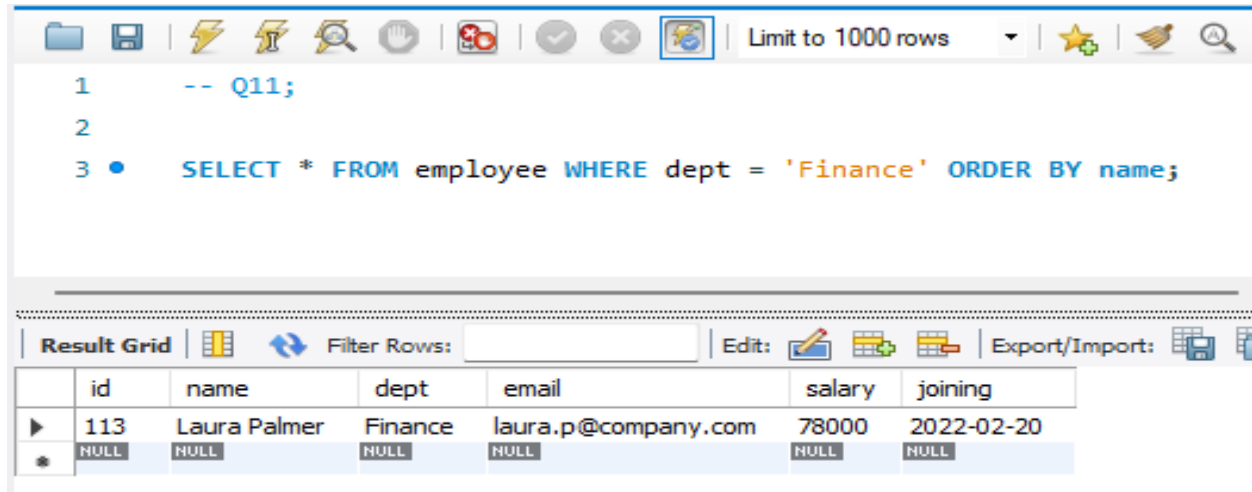
Below the query editor, the 'Result Grid' is displayed. It shows a table with 7 columns: id, name, dept, email, salary, and joining. The results are sorted by salary in descending order.

	id	name	dept	email	salary	joining
▶	105	Diana Prince	Engineering	diana.p@yahoo.com	95000	2024-02-20
	114	Mike Ross	Legal	mike.r@company.com	92000	2021-02-20
	109	Hannah Abbott	Engineering	hannah.a@yahoo.com	88000	2020-02-20
	102	Alice Johnson	Engineering	alice.j@company.com	85000	2022-02-20
	113	Laura Palmer	Finance	laura.p@company.com	78000	2022-02-20
	106	Ethan Hunt	Security	ethan.h@gmail.com	75000	2025-02-20
	112	Kevin Hart	IT	kevin.h@company.com	72000	2023-02-20
	101	Sahil	CSE	sahil@gmail.com	70000	2022-02-20
	115	Nina Simone	Creative	nina.s@company.com	70000	2024-02-20
	111	Jenny Kim	Marketing	jenny.k@gmail.com	64000	2021-02-20
	103	Bob Smith	Sales	bob.s@company.com	62000	2021-02-20
	108	George Miller	Sales	george.m@company.c...	61000	2020-02-20
	107	Fiona Gallagher	Marketing	fiona.g@gmail.com	58000	2022-02-20
	104	Charlie Davis	HR	charlie.d@yahoo.com	55000	2023-02-20
	110	Ian Wright	Support	ian.w@company.com	45000	2021-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q11: Display Finance department employees sorted by name

I filtered with dept = 'Finance'.

Then I used ORDER BY name ASC to sort alphabetically.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
1  -- Q11;
2
3  •  SELECT * FROM employee WHERE dept = 'Finance' ORDER BY name;
```

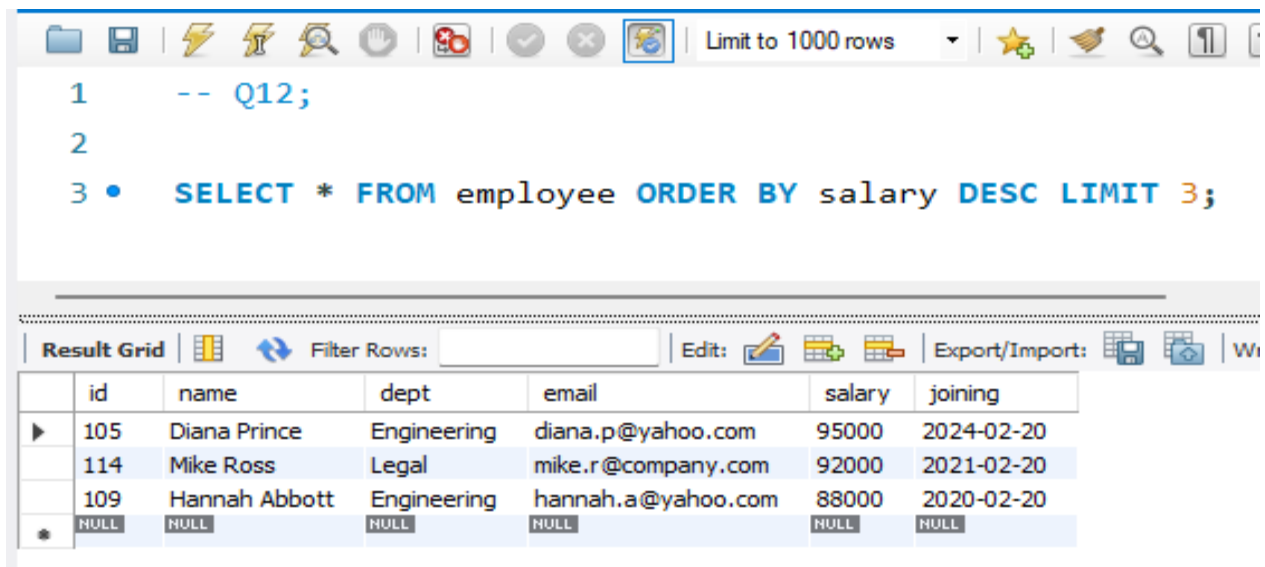
Below the editor is the 'Result Grid' tab, which displays the query results in a table format. The table has columns: id, name, dept, email, salary, and joining. The results show one employee, Laura Palmer, from the Finance department, with a salary of 78000 and a joining date of 2022-02-20. A row with all NULL values is also visible at the bottom of the grid.

	id	name	dept	email	salary	joining
▶	113	Laura Palmer	Finance	laura.p@company.com	78000	2022-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q12: Display the top 3 highest-paid employees

I used ORDER BY salary DESC LIMIT 3.

This gives the top 3 highest-paid employees.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

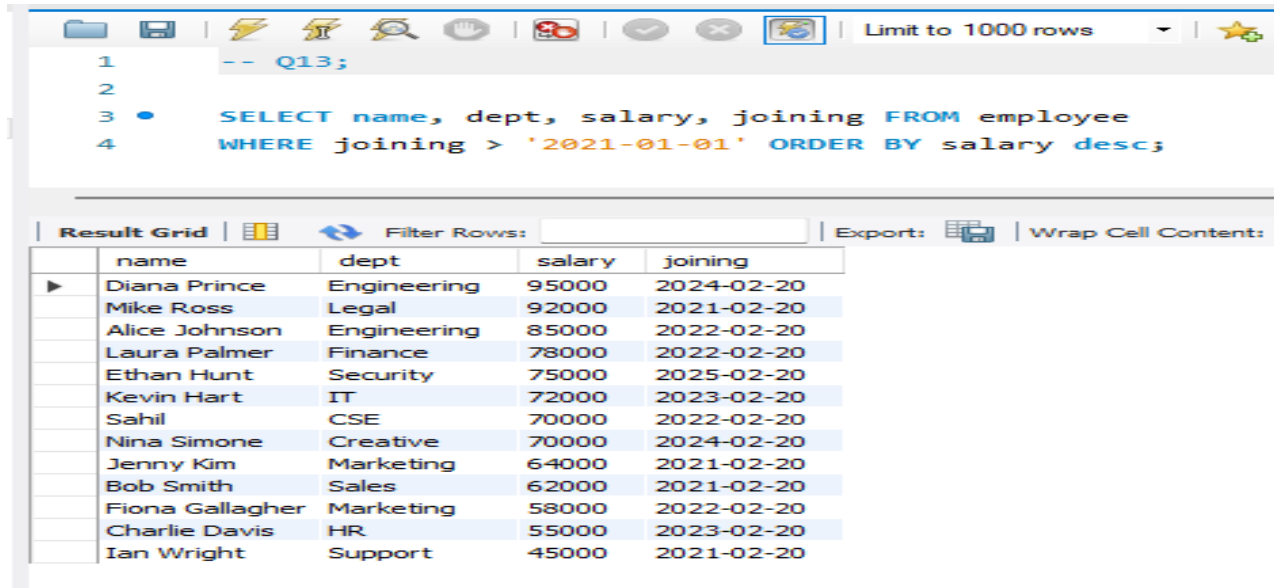
```
1  -- Q12;
2
3  •  SELECT * FROM employee ORDER BY salary DESC LIMIT 3;
```

Below the editor is the 'Result Grid' tab, which displays the query results in a table format. The table has columns: id, name, dept, email, salary, and joining. The results show the top 3 highest-paid employees: Diana Prince (Engineering, salary 95000), Mike Ross (Legal, salary 92000), and Hannah Abbott (Engineering, salary 88000). A row with all NULL values is also visible at the bottom of the grid.

	id	name	dept	email	salary	joining
▶	105	Diana Prince	Engineering	diana.p@yahoo.com	95000	2024-02-20
	114	Mike Ross	Legal	mike.r@company.com	92000	2021-02-20
	109	Hannah Abbott	Engineering	hannah.a@yahoo.com	88000	2020-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL

Q13: Display IT employees hired after 2021 sorted by salary

I filtered IT employees hired after 2021 using WHERE.
Then sorted by salary in descending order for ranking.

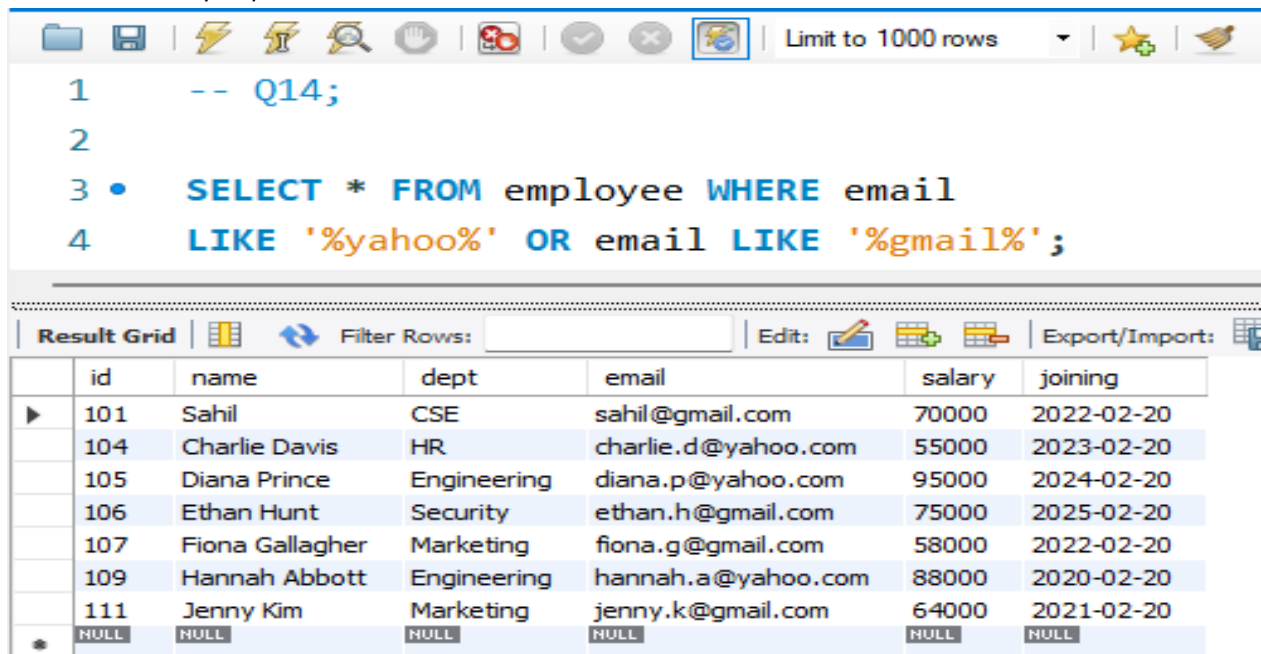


```
1  -- Q13;
2
3  •  SELECT name, dept, salary, joining FROM employee
4     WHERE joining > '2021-01-01' ORDER BY salary desc;
```

	name	dept	salary	joining
▶	Diana Prince	Engineering	95000	2024-02-20
	Mike Ross	Legal	92000	2021-02-20
	Alice Johnson	Engineering	85000	2022-02-20
	Laura Palmer	Finance	78000	2022-02-20
	Ethan Hunt	Security	75000	2025-02-20
	Kevin Hart	IT	72000	2023-02-20
	Sahil	CSE	70000	2022-02-20
	Nina Simone	Creative	70000	2024-02-20
	Jenny Kim	Marketing	64000	2021-02-20
	Bob Smith	Sales	62000	2021-02-20
	Fiona Gallagher	Marketing	58000	2022-02-20
	Charlie Davis	HR	55000	2023-02-20
	Ian Wright	Support	45000	2021-02-20

Q14: Display employees with Gmail or Yahoo email

I used LIKE '%gmail%' OR LIKE '%yahoo%'.
This finds employees with either Gmail or Yahoo emails.



```
1  -- Q14;
2
3  •  SELECT * FROM employee WHERE email
4     LIKE '%yahoo%' OR email LIKE '%gmail%';
```

	id	name	dept	email	salary	joining
▶	101	Sahil	CSE	sahil@gmail.com	70000	2022-02-20
	104	Charlie Davis	HR	charlie.d@yahoo.com	55000	2023-02-20
	105	Diana Prince	Engineering	diana.p@yahoo.com	95000	2024-02-20
	106	Ethan Hunt	Security	ethan.h@gmail.com	75000	2025-02-20
	107	Fiona Gallagher	Marketing	fiona.g@gmail.com	58000	2022-02-20
	109	Hannah Abbott	Engineering	hannah.a@yahoo.com	88000	2020-02-20
	111	Jenny Kim	Marketing	jenny.k@gmail.com	64000	2021-02-20
*	NULL	NULL	NULL	NULL	NULL	NULL